LAPORAN PRAKTIKUM
FUNCTION DAN ARRAY PADA JAVA SCRIPT

OLEH:

Nama	: RachmadPamuji
Nim	: 2025573010103
Kelas	: TI 1\C
Jurusan	: Teknologi Informasi dan Komputer
Dosen	: Muhammad Reza Zulman.S.ST.,M.Sc.

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE

2026

LEMBAR PENGESAHAN

Nama Mata Kuliah	: Praktikum Algoritma Dan Struktur Data
Judul Pratikum	: Praktikum Function Dan Array Pada Java Script
Hari/Tanggal Pratikum	: 08 April 2026
Tanggal Penyerahan	: 14 April 2026
Tempat	: Laboratorium (TIK.106)
Nama Mahasiswa	: Rachmad Pamuji
NIM	: 2025573010103
Program Studi/Kelas	: Teknik Informatika / TI.1C

Mengetahui
Dosen Pengajar,

Muhammad Reza Zulman.S.ST.,M.Sc.

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum yang berjudul *"Praktikum Function dan array pada java script"* dengan baik dan tepat waktu.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan untuk penyempurnaan laporan ini di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada semua pihak yang telah membantu dalam penyusunan laporan ini, baik secara langsung maupun tidak langsung.

Semoga laporan ini dapat bermanfaat bagi pembaca dan memberikan gambaran yang jelas mengenai bahasa pemrograman java script.

DAFTAR ISI

LEMBAR PENGESAHAN i

KATA PENGANTAR ii

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan Praktikum 1

1.3 Pengertian Dan Cara Kerja Dari JavaScript 1

1.4 Sejarah Dari Bahasa Pemrograman JavaScript 2

BAB II PROSEDUR KERJA 3

2.1 Membuat Folder Dan File Package.json 3

2.2 Mengenal Function Dasar 3

2.3 Mengenal Arrow Callback 5

2.4 Mengenal Array Dasar 7

2.5 Mengenal Array Method 9

2.6 Mengenal Rekursi 11

BAB III PENUTUP 13

3.1 Kesimpulan 13

BAB I
PENDAHULUAN

1.1 Latar Belakang

JavaScript sebagai bahasa pemrograman inti web modern menyediakan konsep fundamental seperti Function, Arrow Function, Callback, Array, dan Method Array yang saling terkait untuk mengelola kode secara efisien, memproses data, serta menangani operasi asinkron dalam pengembangan aplikasi interaktif. Function tradisional berfungsi sebagai blok kode reusable untuk modularitas dan mengurangi duplikasi, sementara Arrow Function (diperkenalkan ES6) menawarkan sintaks ringkas dengan binding this leksikal yang konsisten, ideal untuk callback dalam functional programming. Callback sendiri merupakan fungsi yang diteruskan sebagai argumen untuk dieksekusi kemudian, sering dipakai pada event listener, timer, atau pemrosesan asinkron, dan ketika digabung dengan Arrow Function menjadi lebih bersih tanpa masalah konteks this. Array menyimpan data terurut, sedangkan method seperti map(), filter(), reduce(), dan forEach() memanfaatkan callback untuk transformasi data deklaratif, memungkinkan pengolahan data besar secara ringkas dan modern dalam aplikasi web.

1.2 Tujuan Praktikum

Praktikum ini bertujuan untuk memahami konsep dasar,dan perintah perintah dasar yang ada di bahasa pemrograman java script.

- Memahami konsep dasar Function tradisional dan Arrow Function dalam JavaScript.
- Mengimplementasikan Callback untuk mengelola logika kode secara efektif.
- Menguasai Array dan Method Array (map(), filter(), forEach(), reduce()).
- Mengubah pendekatan imperatif menjadi fungsional untuk kode yang lebih readable.
- Meningkatkan modularitas dan performa dalam pengembangan aplikasi web dasar.

1.3 Pengertian Dan Cara Kerja Dari JavaScript

JavaScript adalah bahasa pemrograman tingkat tinggi dan dinamis yang digunakan untuk membuat halaman web menjadi interaktif dan dinamis. Cara kerjanya adalah dengan dieksekusi di sisi klien (browser pengguna) oleh mesin JavaScript yang tertanam di dalamnya. Ketika sebuah halaman web dimuat, kode JavaScript akan diinterpretasikan dan dijalankan oleh browser, memungkinkan fitur-fitur seperti animasi, validasi formulir, dan pembaruan konten secara real-time tanpa perlu memuat ulang halaman secara keseluruhan.

1.4 Sejarah Dari Bahasa Pemrograman JavaScript

JavaScript diciptakan oleh Brendan Eich pada tahun 1995 saat ia bekerja di Netscape Communications. Awalnya, bahasa ini diberi nama Mocha, kemudian LiveScript, sebelum akhirnya diubah menjadi JavaScript. Tujuan utamanya adalah untuk membuat halaman web lebih interaktif dan dinamis di dalam browser Netscape Navigator. Pada tahun 1997, JavaScript distandarisasi sebagai ECMA-262 oleh Ecma International, yang kemudian dikenal sebagai ECMAScript, memastikan interoperabilitas di berbagai browser.

BAB II
PROSEDUR KERJA

2.1 Membuat Folder Dan File Package.json

Untuk membuat folder dan file package.json yang harus kita lakukan adalah dengan membuat folder utama terlebih dahulu Ex.(Praktikum-3) kemdian buka folder tersebut ke visual studio code .selanjutnya buka git bash dan ketik perintah npm init -y maka file package.json akan otomatis terbuat.jika file package.json sudah ada selanjutnya adalah membuat file untuk codingan java script, Ex: nama_file.js

2.2 Mengenal Function Dasar

Function adalah blok kode yang diberi nama dan dapat dipanggil (dieksekusi) kapan saja dan berulang kali. Bayangkan function seperti sebuah mesin atau resep — kamu mendefinisikannya sekali, lalu bisa menggunakannya berkali-kali tanpa perlu menulis ulang kodenya.Mengapa function penting? Prinsip utama pemrograman adalah DRY (Don't Repeat Yourself).Daripada menulis kode yang sama di banyak tempat, cukup bungkus kode itu dalam sebuah function.

Anatomi sebuah function:

- Parameter — Variabel input yang diterima function dari luar. Didefinisikan di dalam kurung ().
- Return value — Nilai yang dikembalikan function setelah selesai dieksekusi, menggunakan kata kunci return.
- Scope — Variabel yang dideklarasikan di dalam function hanya bisa diakses di dalam function itu saja (local scope). Variabel di luar function disebut global scope.

Perlu diingat: setelah JavaScript menemukan kata kunci return, eksekusi function langsungberhenti. Kode di bawah return tidak akan pernah dijalankan.

Contoh Codingan :

```javascript
function sapa(){
console.log('Halo! Selamat datang di praktikum javascript.');
}
sapa ();
sapa();

function sapaNama(nama){
  console.log('Halo, ${nama} !Selamat Belajar.');
}
sapaNama('Budi');
sapaNama('Sitti');

function tambah(angka1,angka2){
  const hasil = angka1 +angka2 ;
  return hasil;
}

const hasilPenjualan =tambah (10,25);
console.log('10 + 25 =',hasilPenjualan);
console.log('7+13 =',tambah(7,13));

function hitung(nilai,pengali =2){
  return nilai * pengali ;
}
console.log(hitung(5));
console.log(hitung(5,4));

const pesanGlobal = 'saya ada dimana saja';

function cekScope(){
  const pesanLokal = 'saya ada didalam fungsi ini';
  console.log(pesanGlobal);
  console.log(pesanLokal);

}
cekScope();
console.log(pesanGlobal);

//LATIHAN 1 KALKULATOR LENGKAP.

console.log('===latihan 1===');

function kurang( a,b){
  return a - b ;
}
function kali (a, b){
  return a * b ;
}
function bagi (a, b){
  if ( b == 0 ) {
    console.log('Error,Tidak bisa dibagi dengan nol.');
    return null ;
  }
  return 1 / b;
}

function tambah(a , b ){
  return a + b ;
}


function kalkulator( a,operator,b){
  switch(operator){
    case '-':
      return kurang(a, b);
      case'x' :
      return kali(a, b);
      case '/':
        return bagi(a, b);
        case '+':
          return tambah(a,b );
          default :
          console.log('error');
          return null;
  }
}
console.log ('10 - 2 =',kalkulator(10, '-' ,5));
console.log (' 3 * 3 =',kalkulator(3,'x',3));
console.log ('12 / 12 =',kalkulator(12,'/',12));
console.log('23 + 40 =',kalkulator(23,'+',40));
console.log('12 / 0 =',kalkulator(12, '/',0));
```

2.3 Mengenal Arrow Callback

Arrow function sebagai callback di JavaScript merupakan sintaks ringkas untuk mendefinisikan fungsi anonim yang secara otomatis mewarisi konteks this dari scope luar, sehingga sangat ideal digunakan dalam array method, event handler, atau asynchronous code seperti setTimeout, Promise, dan fetch. Sintaksnya adalah () => {} untuk tanpa parameter, (param) => nilai untuk satu baris (implicit return), atau (param1, param2) => { kode; } untuk blok lengkap; misalnya dalam array.map(item => item * 2) lebih pendek daripada array.map(function(item) { return item * 2; }) dan menjaga this tetap mengarah ke objek asli tanpa perlu .bind(this). Contoh praktis: setTimeout(() => console.log(this.name), 1000) di dalam class akan menggunakan this dari instance class, sedangkan fungsi biasa akan hilangnya konteks this; kelebihannya termasuk kode lebih bersih dan menghindari masalah binding, tapi hindari untuk object method atau constructor karena tidak punya arguments sendiri dan this tidak bisa diubah.Aturan penulisan arrow function:

- Jika hanya ada satu parameter, kurung bisa dihilangkan: x => x * 2
- Jika body hanya satu baris dan langsung return, kurung kurawal dan return bisa dihilangkan: (a, b) => a + b
- jika body lebih dari satu baris, tetap gunakan kurung kurawal dan return.

Callback adalah sebuah function yang dikirimkan sebagai argumen ke function lain. Function penerima akan memanggil callback tersebut di waktu yang tepat. Ini adalah konsep yang sangat fundamental di JavaScript — hampir semua event handling, timer, dan method array menggunakan callback. Analogi callback: Bayangkan kamu memesan makanan di restoran. Kamu memberikan nomor mejamu (callback) ke pramusaji. Setelah makanan siap (event), pramusaji memanggil nomor mejamu. Kamu tidak tahu kapan tepatnya dipanggil, tapi kamu sudah siapkan respon.

Contoh Codingan :

```javascript
function kuadratBiasa(x){
  return x*x ;
}
const kuadrat =(x)=> {
  return x * x ;
};

const kuadratRingkas = x=> x * x;
console.log ('=== perbandingan tulisan===');
console.log ('biasa   :',kuadratBiasa(5));
console.log ('arrow   :',kuadrat(5));
console.log ('ringkas :',kuadratRingkas(5));

const luas = (panjang,lebar) =>panjang * lebar ;
const salam = (nama, waktu) =>`selamat ${waktu} , ${nama} !`;

console.log ('luas 4x6 : ',luas(4,6));
console.log (salam('budi','pagi'));

function lakukanOperasi(angka,operasionalCallBack){
  console.log('angka awal : ${angka}');
  const hasil = operasionalCallBack(angka);
  console.log(`hasil setelah operasi : $ {hasil}`);
}
console.log('\n===callback===');
lakukanOperasi(7,kuadratRingkas);
lakukanOperasi(10, x => x + 100);
lakukanOperasi(20, function(x){
return x / 2 ;
});
console.log('\n=== setTimeout (Callback) ===');
console.log('Pesan 1: Sebelum timer');
setTimeout(() => {
console.log('Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)');
}, 1000);
console.log('Pesan 2: Setelah mendaftarkan timer');

//LATIHAN 2
 console.log('===Latihan 2===');

const keHurufBesar = (str) => str.toUpperCase();

const tambahSeru = (str) => str + '!!!'

const hitungKata = (str) => str.split(' ').length;

function prosesKalimat(kalimat, transformasiCallback) {
  const hasil = transformasiCallback(kalimat);
  console.log(hasil);
}

const kalimat = 'belajar javascript itu menyenangkan';

prosesKalimat(kalimat, keHurufBesar);
prosesKalimat(kalimat, tambahSeru);
prosesKalimat(kalimat, hitungKata);
```

Hasilnya : Tiga cara penulisan function, yaitu function biasa, arrow function, dan function ringkas, sebenarnya menghasilkan output yang sama. Misalnya saat memanggil kuadrat(5), ketiganya tetap memberikan hasil yang sama, sehingga bisa dibilang ketiga cara tersebut setara. Selain itu, jika diperhatikan pada penggunaan setTimeout, urutan output yang muncul adalah Pesan 1, kemudian Pesan 2, lalu setelah menunggu sekitar 1 detik baru muncul Pesan 3. Hal ini menunjukkan bahwa JavaScript tidak menunggu proses timer selesai, melainkan langsung melanjutkan eksekusi program. Konsep ini dikenal sebagai asynchronous. Untuk menangani proses seperti ini biasanya digunakan callback, yaitu function yang akan dijalankan setelah suatu proses selesai. Pada pemanggilan lakukanOperasi, terlihat bahwa hasil output dapat berbeda tergantung function (callback) yang diberikan sebagai argumen, sehingga dengan mengganti callback, hasil yang diperoleh juga akan berubah.

2.4 Mengenal Array Dasar

Array adalah struktur data yang menyimpan kumpulan nilai dalam satu variabel, berurutan dan terindeks. Bayangkan array seperti sebuah rak buku bernomor — setiap buku (elemen) memiliki
posisi (indeks) yang dimulai dari angka 0.

Contoh: const buah = ['Apel', 'Mangga', 'Jeruk']

- buah[0] → 'Apel' (indeks pertama = 0)
- buah[1] → 'Mangga' (indeks kedua = 1)
- buah[2] → 'Jeruk' (indeks ketiga = 2)
- buah.length → 3 (jumlah elemen)

Array di JavaScript bersifat dinamis — ukurannya bisa berubah sewaktu-waktu. Array juga bisa
menyimpan tipe data yang berbeda-beda dalam satu array (meskipun praktik terbaik adalah menyimpan satu jenis data per array). Method bawaan array yang sering digunakan :

- push(x) — tambah elemen x di akhir array
- pop() — hapus dan kembalikan elemen terakhir
- unshift(x) — tambah elemen x di awal array
- shift() — hapus dan kembalikan elemen pertama
- splice(i, n) — hapus n elemen mulai indeks i
- slice(a, b) — salin elemen dari indeks a sampai sebelum b (non-destructive)
- indexOf(x) — cari indeks elemen x (-1 jika tidak ditemukan)
- includes(x) — cek apakah x ada di array (mengembalikan boolean),Contoh Codingan :

```javascript
const mahasiswa = ["Budi", "Siti", "Ahmad", "Rina"];
const nilai = [85, 92, 78, 95, 88];
console.log("=== Array Awal ===");
console.log("Mahasiswa:", mahasiswa);
console.log("Nilai :", nilai);
console.log("Jumlah mahasiswa:", mahasiswa.length);

console.log("\n=== Akses Elemen ===");
console.log("Elemen pertama :", mahasiswa[0]);
console.log("Elemen ketiga :", mahasiswa[2]);
console.log("Elemen terakhir:", mahasiswa[mahasiswa.length - 1]);

mahasiswa[1] = "Siti Rahayu";
console.log("\nSetelah diubah:", mahasiswa);

console.log("\n=== Manipulasi Array ===");
mahasiswa.push("Doni");
console.log("Setelah push :", mahasiswa);
mahasiswa.unshift("Zahra");
console.log("Setelah unshift :", mahasiswa);
const dihapusAkhir = mahasiswa.pop();
console.log("Dihapus (pop) :", dihapusAkhir);
console.log("Setelah pop :", mahasiswa);
const dihapusAwal = mahasiswa.shift();
console.log("Dihapus (shift) :", dihapusAwal);
console.log("Setelah shift :", mahasiswa);

console.log("\n=== Pencarian ===");
console.log("Indeks Ahmad :", mahasiswa.indexOf("Ahmad"));
console.log("Ada Rina? :", mahasiswa.includes("Rina"));
console.log("Ada Budi? :", mahasiswa.includes("Budi"));

const sebagian = nilai.slice(1, 4);
console.log("\nNilai asli :", nilai);
console.log("Slice [1,4] :", sebagian);

//latihan 3
console.log('===Latiahn 3===');
const daftarBelanja = ['Beras', 'Gula', 'Susu', 'Telur', 'Minyak'];

console.log('=== Daftar Belanja ===');
for (let i = 0; i < daftarBelanja.length; i++) {
  console.log(`${i + 1}. ${daftarBelanja[i]}`);
}

daftarBelanja.push('Tepung', 'Garam');
console.log('\n=== Setelah Menambah 2 Item ===');
console.log(daftarBelanja);

const itemDihapus = daftarBelanja.shift();
console.log(`\nItem yang dihapus: ${itemDihapus}`);

const adaSusu = daftarBelanja.includes("Susu");
console.log(
  `\nApakah 'Susu' ada di daftar? ${adaSusu ? 'Ya, Susu tersedia' : 'Tidak, Susu tidak ada'}`,
);

console.log(`\nJumlah item akhir di daftar: ${daftarBelanja.length} item`);
```

2.5 Mengenal Array Method

Array method pada JavaScript merupakan fungsi-fungsi bawaan yang memudahkan manipulasi data dalam array, dibagi menjadi mutator yang mengubah array asli seperti push() untuk menambah elemen di akhir, pop() untuk menghapus elemen akhir, unshift() untuk menambah di awal, shift() untuk menghapus dari awal, serta splice() untuk menghapus, mengganti, atau menambah elemen di posisi tertentu. Selain itu, ada accessor method yang tidak mengubah array asli, seperti slice() untuk mengambil potongan array, concat() untuk menggabungkan array, join() untuk mengubah array menjadi string dengan pemisah tertentu (misalnya ['a','b'].join('-') menghasilkan "a-b"), dan toString() untuk mengonversi ke string dengan koma sebagai pemisah. Method iterator modern ES6+ yang paling powerful meliputi map() untuk mentransformasi setiap elemen (seperti mengkuadratkan semua angka), filter() untuk memilih elemen yang memenuhi kondisi, reduce() untuk mengakumulasi nilai seperti menghitung total, forEach() untuk menjalankan fungsi per elemen, find() untuk mencari elemen pertama yang cocok, serta sort() untuk mengurutkan array (dengan fungsi perbandingan agar tidak berurut secara string secara default).

sebuah callback function sebagai argumen, dan secara otomatis memanggil callback untuk setiap elemen array.

- `forEach(callback)` — Menjalankan callback untuk setiap elemen. Tidak mengembalikan nilai apapun. Digunakan hanya untuk efek samping (side effects) seperti mencetak atau menyimpan ke database.
- `map(callback)` — Membuat array baru dengan menerapkan callback ke setiap elemen. Array asli tidak berubah. Digunakan untuk mentransformasi setiap elemen.
- `filter(callback)` — Membuat array baru yang hanya berisi elemen yang memenuhi kondisi (callback mengembalikan true). Array asli tidak berubah. Digunakan untuk menyaring elemen.
- `reduce(callback, nilaiAwal)` — Mereduksi seluruh array menjadi satu nilai tunggal. Callback menerima dua argumen: accumulator (nilai yang terakumulasi) dan currentValue (elemen saat ini).

Ingat: map dan filter selalu menghasilkan array baru, sedangkan reduce menghasilkan satu nilai (angka, string, object, dll).

Contoh Codingan :

```javascript
const nilaiMahasiswa = [75, 90, 55, 82, 68, 95, 48, 77];

console.log("=== forEach: Tampilkan Semua Nilai ===");
nilaiMahasiswa.forEach((nilai, indeks) => {
  console.log(` Mahasiswa-${indeks + 1}: ${nilai}`);
});

const gradeHuruf = nilaiMahasiswa.map((nilai) => {
  if (nilai >= 90) return 'A';
  if (nilai >= 80) return 'B';
  if (nilai >= 70) return 'C';
  if (nilai >= 60) return 'D';
  return "E";
});
console.log('\n=== map: Nilai ke Grade ===');
console.log('Nilai :', nilaiMahasiswa);
console.log("Grade :", gradeHuruf);

const nilaiLulus = nilaiMahasiswa.filter((nilai) => nilai >= 60);
const nilaiGagal = nilaiMahasiswa.filter((nilai) => nilai < 60);
console.log('\n=== filter: Lulus dan Tidak Lulus ===');
console.log('Semua nilai :', nilaiMahasiswa);
console.log('Nilai lulus :', nilaiLulus);
console.log('Nilai gagal :', nilaiGagal);

const totalNilai = nilaiMahasiswa.reduce((akumulator, nilai) => {
  return akumulator + nilai;
}, 0);
const rataRata = totalNilai / nilaiMahasiswa.length;
console.log('\n=== reduce: Statistik Nilai ===');
console.log('Total nilai :', totalNilai);
console.log('Rata-rata :', rataRata.toFixed(2));
const rataRataNilaiLulus = nilaiMahasiswa
  .filter((nilai) => nilai >= 60)
  .reduce((acc, val, idx, arr) => {
    return acc + val / arr.length;
  }, 0);
console.log('\n=== Method Chaining ===');
console.log('Rata-rata nilai lulus:', rataRataNilaiLulus.toFixed(2));

//latihan 4
console.log('===Latihan 4===');
const produk = [
  { nama: "Laptop", harga: 8500000, stok: 5 },
  { nama: "Mouse", harga: 150000, stok: 0 },
  { nama: "Keyboard", harga: 450000, stok: 12 },
  { nama: "Monitor", harga: 3200000, stok: 0 },
  { nama: "Headset", harga: 350000, stok: 8 },
];

const produkTersedia = produk.filter((p) => p.stok > 0);
console.log("=== Produk Tersedia ===");
console.log(produkTersedia);

const namaProduk = produkTersedia.map((p) => p.nama);
console.log("\n=== Nama Produk Tersedia ===");
console.log(namaProduk);

const totalInventaris = produkTersedia.reduce(
  (total, p) => total + p.harga * p.stok,
  0,
);
console.log("\n=== Total Nilai Inventaris ===");
console.log(`Rp${totalInventaris.toLocaleString("id-ID")}`);

console.log("\n=== Daftar Semua Produk ===");
produk.forEach((p) => {
  if (p.stok > 0) {
    console.log(
      `[TERSEDIA] ${p.nama} - Rp${p.harga.toLocaleString("id-ID")} (${p.stok} unit)`,
    );
  } else {
    console.log(`[HABIS] ${p.nama} - Rp${p.harga.toLocaleString("id-ID")}`);
  }
});
```

2.6 Mengenal Rekursi

Rekursi pada JavaScript merupakan teknik pemrograman di mana sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan masalah yang dapat dipecah menjadi sub-masalah serupa, dengan dua komponen utama yaitu base case (kondisi dasar) yang menghentikan pemanggilan agar tidak menjadi infinite loop, dan recursive case yang memanggil fungsi tersebut dengan input yang lebih kecil hingga mencapai kondisi dasar. Misalnya, fungsi faktorial dapat ditulis sebagai function faktorial(n) { if (n <= 1) return 1; return n * faktorial(n - 1); }, di mana faktorial(5) akan menghitung secara bertahap: 5 * faktorial(4) hingga mencapai 1, menghasilkan 120. Contoh lain adalah penjumlahan angka dari 1 hingga n melalui function sum(num) { if (num === 0) return 0; return num + sum(num - 1); }, sehingga sum(4) menghasilkan 10 melalui proses 4 + 3 + 2 + 1 + 0. Teknik ini sangat cocok untuk masalah berstruktur pohon seperti Fibonacci atau traversal array multidimensi karena lebih deklaratif mirip loop, tetapi berisiko menyebabkan stack overflow jika kedalaman rekursi terlalu besar, sehingga sebaiknya gunakan loop iteratif untuk kasus sederhana agar lebih efisien.masalah yang lebih kecil secara bertahap, sampai mencapai kondisi berhenti.

Setiap function rekursif harus memiliki dua bagian:

- Base case (kondisi berhenti) — kondisi di mana function tidak lagi memanggil dirinya sendiri. Tanpa ini, rekursi akan berjalan selamanya dan menyebabkan error Maximum call stack exceeded.
- Recursive case — bagian di mana function memanggil dirinya sendiri dengan input yang lebih kecil/sederhana.

Contoh: Menghitung faktorial dari n (n!):

- 5! = 5 × 4! = 5 × 4 × 3! = 5 × 4 × 3 × 2! = 5 × 4 × 3 × 2 × 1! = 120
- Base case: faktorial(1) = 1
- Recursive case: faktorial(n) = n × faktorial(n-1)

Rekursi sangat penting dalam Struktur Data dan Algoritma karena banyak algoritma pada pohon (tree), graf (graph), dan pencarian (searching) yang paling mudah diimplementasikan secara rekursif.

Contoh Codingan :

```
function faktororial(n) {
  //base case : berhenti saat n=1
  if (n <= 1) return 1;

  return n * faktororial(n - 1);
}
console.log('===faktorial===');
console.log('0! =', faktororial(0));
console.log('1! =', faktororial(1));
console.log('5! =', faktororial(5));
console.log('7! =', faktororial(7));

function fibonacci(n) {
  if (n === 0) return 0;
  if (n === 1) return 1;

  return fibonacci(n - 1) + fibonacci(n - 2);
}
console.log('\n=== Fibonacci ===');
for (let i = 0; i <= 8; i++) {
  process.stdout.write(fibonacci(i) + ' ');
}
console.log("");

function jumlahArray(arr, indeks = 0) {
  if (indeks === arr.length) return 0;

  return arr[indeks] + jumlahArray(arr, indeks + 1);
}
const angka = [3, 7, 2, 9, 5];
console.log('\n=== Jumlah Array dengan Rekursi ===');
console.log('Array :', angka);
console.log('Jumlah :', jumlahArray(angka));

function countdown(n) {
  if (n < 0) {
    console.log('Selesai!');
    return;
  }
  console.log(n);
  countdown(n - 1);
}
console.log('\n=== Countdown Rekursif ===');
countdown(5);
```

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan materi praktikum di atas dapat di simpulkan bahwa array dan function di java script adalah sebagai dasar yang kuat untuk membuat kita bisa menghasilkan sebuah aplikasi. array dan function sangat penting karena memungkinkan pengelolaan data secara efisien dan modularitas kode untuk membangun aplikasi web dinamis. Array berfungsi sebagai struktur data khusus untuk menyimpan koleksi nilai yang tertata (seperti daftar pengguna, produk, atau hasil API) dengan indeks numerik mulai dari 0, memudahkan operasi seperti map/filter/reduce untuk pemrosesan data tanpa mengubah array asli, sehingga optimal untuk performa memori dan kecepatan akses dibanding objek biasa. Function memungkinkan kode reusable yang dapat dipanggil berulang kali dengan parameter, mendukung callback/arrow function untuk event handling, async operations, dan array methods, serta sebagai first-class citizen yang bisa disimpan di array/objek untuk pola seperti higher-order function atau closure, sehingga mengurangi duplikasi kode dan meningkatkan maintainability program. Keduanya saling melengkapi: array menyediakan wadah data terstruktur, sementara function menyediakan logika pengolahan, esensial untuk aplikasi nyata seperti manajemen daftar todo, filtering data API, atau animasi DOM.